

Linked List & Hashing Answers

March 30, 2025

1 Linked List

1. Doubly Linked List struct:

```
struct ListNode {
    int val;
    struct ListNode *prev;
    struct ListNode *next;
};
```

2. printList() function:

```
void printList(struct ListNode *head) {
    struct ListNode *curr = head;
    while (curr != NULL) {
        printf("%d -> ", curr->val);
        curr = curr->next;
    }
    printf("NULL\n");
}
```

3. swap() function:

```
// swap first two nodes
struct ListNode* swap(struct ListNode *head) {
    // assuming size >=2
    struct ListNode* next = head->next;
    struct ListNode* nextnext = next->next;

    // modify connections
    head->next = nextnext;
    next->next = head;
    return next;
}
```

2 Hashing

1. Checking if an element exists has average complexity of $O(1)$, also known as constant time. This requires the assumption that the hashFunction spreads elements out mostly uniformly (so there are minimal collisions) and the hashFunction is efficient (using bitwise operations/modulo) so that keys can be hashed in constant time. The steps for retrieval are:
 - (a) hash the key to give us a hash code – assume hashFunction is $O(1)$
 - (b) get the index from the hash code – using modulo/bitwise, $O(1)$
 - (c) access the element at that index – array access is $O(1)$

Thus, HashMap search() is $O(1)$ overall under these assumptions.

2. HashMap collisions occur when two different keys hash to the same index in the hash table. To handle collisions, we can use **separate chaining**. Each index in our hash table stores a **linked list** of entries. When a collision occurs, the new entry is added to the list at that index. Retrieval involves checking the list at that index, which can be $O(N)$ in the worst case (imagine if every key hashes to the same index). Average performance can still remain $O(1)$.
3. HashMap insert():

```
#include <stdio.h>
#include <stdlib.h>

#define TABLE_SIZE 10

typedef struct {
    int key;
    int value;
} HashEntry;

HashEntry *hashTable[TABLE_SIZE] = {NULL};

// Simple hash function
int hashFunction(int key) {
    return key % TABLE_SIZE;
}

void insert(int key, int value) {
    // 1) hash the key
    int index = hashFunction(key);

    // 2) create a HashMap entry with appropriate key and value
    HashEntry *entry = (HashEntry *)malloc(sizeof(HashEntry)
    );
```

```
entry->key = key;
entry->value = value;

// 3) store the entry in the HashMap
if (hashTable[index] == NULL) {
    // no collision, simply store the entry
    hashTable[index] = entry;
} else {
    // handle collision here (but we won't in this
    // example):
    hashTable[index] = entry;
}
}
```

Again, assuming a good hashFunction and appropriate size of hash table, insertion is $O(1)$, constant-time. Performance can degrade to $O(N)$ when there are frequent collisions, causing multiple entries to be stored in the same table index. This increases the time taken to find an entry or insert a new one.